

KAVACH AI

Independent Work Intelligence & Protection for Gig Workers

Product, AI, Architecture, Scope & Hackathon Source of Truth

North-star promise

Platforms have a record of every trip, decision, payment and penalty. Kavach gives the worker an independent record too—and uses AI to understand what happened, detect what does not add up, predict better choices, and help the worker act.

Prepared for First Commit — Bharat Builds Tour • Ship It target

Version 1.0 • 15 September 2026

1. Executive Overview

Kavach AI is an AI-native worker intelligence layer for India's gig economy. It is designed for workers whose income and access to work are mediated by platform software. The product does not attempt to replace a platform or act as a court. Instead, it creates an independent worker-side understanding of earnings, trips, incentives, incidents and platform decisions, then uses deterministic calculations and agentic AI to help the worker interpret and act on that evidence.

Core thesis

Kavach is not “an AI chatbot for gig workers.” It is an independent evidence and decision layer that answers four questions: What happened? Did the numbers add up? What is likely to happen next? What should I do?

1.1 Product pillars

Pillar	Purpose	Primary AI contribution	Primary output
Understand	Turn messy worker data into structured reality.	Multimodal extraction, summarization, retrieval.	Worker timeline and earnings picture.
Reconstruct	Explain what happened during a trip or incident.	Forensic reasoning across events and documents.	Evidence-backed event narrative.
Audit	Find discrepancies, anomalies and lost income.	Anomaly detection + agent investigation.	Potential discrepancy / supported / insufficient evidence.
Predict & Optimize	Help the individual worker make better choices.	Worker Twin + simulation + scenario reasoning.	Recommended shift, order or action.
Protect	Help when something goes wrong.	Policy retrieval + case synthesis.	Evidence-backed review/appeal package.

1.2 The flagship user experience

A worker sees a ₹350 late-delivery penalty. Rather than merely drafting a complaint, Kavach reconstructs the trip, identifies that 7 of 10 SLA minutes were consumed before package handoff, checks traffic/context, evaluates the payout discrepancy, cites the evidence, explains the confidence level, and creates a structured review request. The same evidence can then inform earnings analytics and future recommendations.

Winning-demo moment

“The platform says the rider was late. Kavach reconstructs the trip and shows that most of the SLA was already consumed before the rider left the store.”

2. Product Strategy for First Commit

The project is explicitly optimized for the stated First Commit judging rubric: real impact, meaningful AWS usage, visible learning, a working end-to-end implementation, and a three-minute recorded demo. The project should target the Ship It track because the cloud architecture itself becomes part of the score.

Judging criterion	Kavach answer	Evidence to show
Idea & Impact	Addresses information asymmetry in gig work: earnings, automated decisions, waiting, penalties, deactivation risk and worker agency.	Concrete worker case and quantified outcome.
Built on AWS	AWS is the product substrate, not decoration.	Amplify, Cognito, S3, Lambda/API Gateway, DynamoDB, OpenSearch, Bedrock, Strands, Step Functions/EventBridge, Cedar.

Learning	Team learns agent orchestration, multimodal extraction, retrieval, policy-as-code, serverless workflow and deployment.	Architecture retrospective in write-up/video.
Execution	One investigation path works end-to-end.	Upload → extract → reconstruct → detect → explain → act.
Demo video	One worker, one painful event, one AI investigation.	Live-looking, pre-seeded scenario with visible evidence and result.

Scope principle

Build breadth in the product model, but concentrate reliability on one complete vertical slice. The product can expose multiple capabilities because they share the same evidence graph and worker ledger.

3. Problem Definition

Gig work creates a strong information asymmetry. The platform sees granular operational events; the worker often sees only a final payout, rating, warning or deactivation message. A worker may therefore be unable to explain why income changed, whether a deduction was consistent with the applicable rules, or what evidence exists when a decision is disputed.

3.1 Primary user

Primary persona: an urban gig worker performing delivery or other location-based platform work. The hackathon demo should focus on a delivery worker because trips, waiting, incentives, expenses, routes and penalties create highly legible data and a visually understandable narrative.

3.2 Secondary personas

- Worker advocate/support agent: needs a case summary and evidence bundle without exposing unnecessary raw telemetry.
- Authorized reviewer: needs policy context, evidence coverage and the worker's requested action.
- Aggregated research/worker-voice view: sees anonymized patterns, not individual private records.

3.3 Jobs to be done

Job	Current pain	Kavach outcome
Explain my earnings	Platform payout hides costs, waiting and missing incentives.	Real-earnings breakdown and causal explanation.
Challenge a penalty	Worker lacks evidence tying the penalty to what actually happened.	Evidence-backed trip reconstruction and case package.
Understand a restriction	Automated decisions may be opaque or difficult to contest.	Decision forensics + relevant policy + recommended review action.
Choose how to work	Worker optimizes on incomplete information.	Personalized scenario recommendations.
Recover after an incident	Evidence and documents are fragmented after accidents.	Structured incident/claim evidence bundle.

4. Product Principles

Evidence before conclusion

Every important AI conclusion should link to underlying evidence or clearly state what is missing.

AI reasons; code verifies

The model interprets documents and patterns. Deterministic code handles arithmetic, thresholds, state transitions and authorization.

Uncertainty is a feature

Kavach must distinguish supported, potential discrepancy, high-risk dispute and insufficient evidence.

Worker-owned view

The product should feel like a worker-side source of truth, not a surveillance system.

Privacy by design

Raw telemetry is sensitive. Store the minimum necessary, use coarse location where possible, and strictly control exports.

No fake legal certainty

Kavach surfaces relevant law/policy and prepares evidence; it does not claim to adjudicate legality.

One evidence graph, many experiences

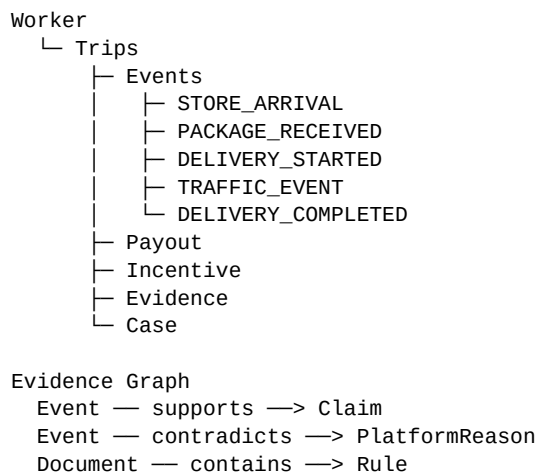
Earnings, forensics, safety and protection are different views over shared data.

Ship one vertical slice

Secondary modules must never compromise the primary investigation workflow.

5. Core Domain Model: Worker Ledger + Evidence Graph

The product's most important technical abstraction is an independent Worker Ledger. It stores meaningful worker events and links them to artifacts. The Evidence Graph connects those events to documents, calculations, policies, observations and conclusions.



Calculation — derives_from —> Events
AgentFinding — cites —> Evidence

5.1 Why this architecture matters

The graph allows every surface of the product to reuse the same underlying facts. An earnings discrepancy can cite a payout document and trip events; the same trip events can support a dispute; the same history can feed the Worker Twin; the same patterns can contribute to anonymized collective intelligence.

6. Feature Specifications

6.1 Kavach Financial — True Earnings

Purpose: replace headline platform earnings with a transparent estimate of real earnings. Inputs include platform payout, working time, fuel, maintenance/depreciation assumptions and unpaid waiting. Output is gross payout, worker costs, net effective earnings and effective hourly earnings. AI role: identify and normalize inputs from screenshots/docs, explain the causal drivers of changes, and summarize patterns. Deterministic role: arithmetic and cost formulas. Hackathon acceptance: at least one seeded weekly dataset produces a coherent breakdown and explains why the week changed.

6.2 Kavach PayCheck — Payout & Incentive Audit

Purpose: compare expected vs actual compensation. Inputs include trip records, incentive rules, payout statements and worker evidence. AI role: extract incentive conditions, match trips to eligibility and investigate anomalies. Deterministic role: count trips, calculate distance/time thresholds, reconcile expected and actual values. Output: discrepancy amount, confidence, evidence list and suggested next step.

6.3 Kavach Proof — Trip Reconstruction

Purpose: independently reconstruct what happened during a trip. Inputs: timestamps, coarse location, trip metadata, screenshots and optional context such as weather/traffic. AI role: generate a human-readable narrative and identify causal sequences. Deterministic role: calculate elapsed time, SLA consumption and observed speeds. Output: timeline, SLA budget, evidence coverage and assessment.

6.4 Counterfactual Trip Simulator

Purpose: answer whether a target SLA was realistically achievable under observed conditions. AI role: choose relevant variables and explain scenarios. Deterministic role: simulate remaining time and required speed under alternative assumptions. Output: baseline, no-delay scenario, low-traffic scenario and feasibility classification. This is a flagship “AI + math” capability.

6.5 Decision Review / Deactivation Shield

Purpose: help a worker understand an account action and prepare for human review. Inputs: platform message, available history, route anomalies, ratings and worker evidence. AI role: reconstruct likely contributing factors and identify contradictions. Policy agent role: retrieve relevant platform/regulatory text. Output: assessment, supporting evidence, contradictory evidence, missing evidence, recommended request.

6.6 Kavach Resolve — Evidence-backed Case Builder

Purpose: turn findings into a structured review package. AI role: synthesize evidence and produce a neutral case narrative. Deterministic role: include exact amounts, dates and event references. Output: issue, claimed amount, observed evidence, requested action, attachments, uncertainty statement.

6.7 Worker Twin — Personal Work Intelligence

Purpose: build a compact representation of the worker's recurring patterns: earnings, locations, wait times, incentives, expenses, work windows and preferences. AI role: personalize explanations and recommendations. Deterministic role: compute historical statistics and simulation outputs. Output: recommendations such as best shift window, avoid-zone, target earning plan and confidence.

6.8 GigOpt — Work Optimization

Purpose: recommend which work opportunities are likely to maximize net/risk-adjusted earnings. Inputs: order economics, distance, expected wait, fuel, historical location patterns, active incentives and worker preferences. AI role: select relevant factors and explain trade-offs. Deterministic role: scoring and scenario calculations. Output: recommended option + rationale + expected net range.

6.9 Safety Intelligence

Purpose: identify potentially high-risk work patterns from duration, workload, weather/context and incident history. AI role: synthesize signals and explain them in plain language. Output should be advisory, not diagnostic. This is stretch scope after the primary workflow works.

6.10 ClaimKavach — Incident Evidence

Purpose: organize an accident or incident into a structured evidence pack. AI role: multimodal extraction from images/PDFs/voice notes and missing-document identification. Output: timeline, documents, missing evidence and a neutral incident summary.

6.11 Policy Intelligence

Purpose: answer rights/process questions using retrieved source documents. Inputs: Karnataka gig-worker law, platform policy, insurance terms, incentive rules and curated guidance. AI role: retrieval-grounded explanation. Output must quote/cite the source document internally and clearly separate source text from interpretation.

6.12 Worker Voice / Collective Intelligence

Purpose: surface anonymized patterns across workers. AI role: cluster recurring issues and generate hypotheses. Output: aggregated patterns such as stores with unusual waiting or recurring incentive discrepancies. This must never expose individual worker records.

7. AI System Architecture

AI is the reasoning layer, not the authority layer

Kavach intentionally separates probabilistic model reasoning from deterministic calculations, evidence storage, state transitions and authorization. This is both a product-trust decision and a strong engineering story.

7.1 Agent roles

Agent	Primary responsibility	Tools / inputs	Output contract
-------	------------------------	----------------	-----------------

Supervisor / Orchestrator	Understand request, select specialists, merge findings.	Worker context, agent registry.	Plan + final synthesis.
Earnings Agent	Reconcile payouts, incentives, costs.	Payout tools, ledger queries, calculation service.	Findings + amounts + evidence IDs.
Forensics Agent	Reconstruct trip/incident causality.	Trip events, telemetry summaries, documents.	Timeline + causal findings + confidence.
Policy Agent	Retrieve and explain relevant rules.	OpenSearch / knowledge base.	Relevant provisions + source references.
Worker Twin Agent	Run personalized scenario analysis.	Historical aggregates + simulator.	Recommendation + scenario range + confidence.

The number of agents should stay small. Do not create agents merely to increase architecture complexity. Every agent must have a distinct responsibility, constrained tool access and a structured output contract.

7.2 Strands implementation model

Strands is the preferred orchestration layer because it is an SDK/library for building tool-using agents in an application process. Amazon Bedrock is the default model provider in Strands documentation, while AWS also documents multi-agent patterns with role-defined supervisors and collaborators. Use Strands for the project's visible "agent" implementation while keeping tool boundaries explicit. [R1][R2]

```
User question
-> Supervisor agent
    -> call earnings tool
    -> call evidence tool
    -> call policy retrieval tool
    -> call simulator tool
-> synthesize structured result
-> render evidence + recommended action
```

7.3 Model routing

Task	Model strategy	Reason
Image/document extraction	Fast multimodal Bedrock model.	Low latency/cost; structured fields.
Routine worker explanation	Efficient general-purpose model.	Lower cost for high-volume interactions.
Complex multi-source investigation	Stronger reasoning model.	Better synthesis over conflicting evidence.
Simulation/calculation	No LLM.	Deterministic and reproducible.
Authorization	No LLM.	Cedar must decide access.

8. AI Reliability & Evaluation

The project should treat evaluation as a first-class product feature. The AI must be able to say when evidence is insufficient.

Metric	Definition	Target for hackathon prototype
Extraction accuracy	Correct structured fields from seeded screenshots/docs.	≥ 90% on curated test set.
Evidence citation coverage	Major claims linked to evidence IDs.	100% for primary demo path.
Arithmetic correctness	Deterministic outputs match expected fixtures.	100% on tested fixtures.
Dispute classification consistency	Same input yields stable category.	≥ 90% across repeat runs.
Unsupported-claim rate	Claims not supported by evidence.	Near zero on demo scenarios.
Tool-call validity	Agent invokes tools with valid schemas.	≥ 95% in test set.

8.1 Structured finding schema

```
{
  "finding_id": "F-18421-01",
  "type": "potential_discrepancy",
  "claim": "Late-delivery penalty may be inconsistent with observed timeline",
  "confidence": 0.94,
  "evidence_ids": ["E-1", "E-2", "E-7"],
  "missing_evidence": ["platform_internal_dispatch_timestamp"],
  "calculation_ids": ["C-18421-01"],
  "recommended_action": "request_human_review"
}
```

9. Policy, Legal and Governance Layer

Kavach must not present itself as an automated legal adjudicator. It is an evidence and decision-support product. The policy layer retrieves source material and helps the worker understand applicable processes, obligations or review mechanisms. Any legal interpretation should be framed as informational and should point back to source text.

The Karnataka Platform Based Gig Workers (Social Security and Welfare) Act, 2025 includes a provision on transparency in automated monitoring and decision-making systems. It requires aggregators/platforms to inform gig workers, in simple language and in Kannada, English or another known Eighth Schedule language, about the procedure for seeking information on automated monitoring and decision-making parameters that affect working conditions, including fares, earnings and customer feedback. [R3]

Important design implication

Kavach's multilingual Policy Agent and Decision Review experience directly support the problem of workers understanding automated decision-making. However, the product should never claim that a Cedar evaluation itself establishes a statutory violation.

9.1 Authorization with Cedar

Use Cedar to enforce application authorization: who may read raw telemetry, who may export evidence, which support roles can view anonymized summaries, and which actions require explicit worker consent. Cedar returns ALLOW or DENY decisions and separates authorization logic from application business logic. [R4]

Example policy intent:

- Worker can read own ledger.
- Worker can export own evidence package.
- Support role can view assigned case metadata.
- Support role cannot read raw GPS without explicit authorization.
- Aggregated analytics can read anonymized statistics only.

10. Data & Storage Specification

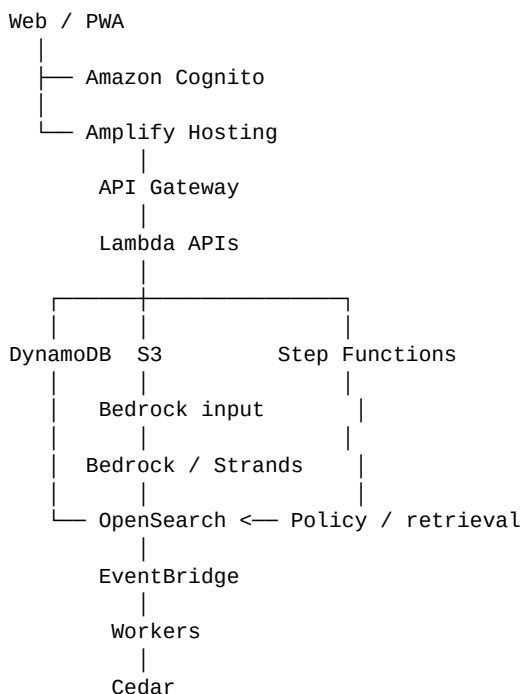
Entity	Purpose	Primary fields	Storage
Worker	Identity and preferences.	worker_id, locale, goals, authorized platforms.	DynamoDB
Trip	Trip-level record.	trip_id, timestamps, SLA, store, destination, payout.	DynamoDB
Event	Immutable/append-oriented timeline event.	event_id, type, time, coarse location, source.	DynamoDB / S3 for bulky evidence

Artifact	Screenshot/PDF/photo/audio.	artifact_id, S3 key, type, hash, metadata.	S3
Payout	Actual compensation.	gross, net, deductions, incentives.	DynamoDB
Calculation	Deterministic derived value.	formula_id, inputs, result, timestamp.	DynamoDB
Finding	AI-produced structured conclusion.	claim, confidence, evidence_ids, missing_evidence.	DynamoDB
Case	Dispute/review lifecycle.	case_id, issue, status, requested action, evidence.	DynamoDB
PolicyDocument	Source text for retrieval.	source_id, jurisdiction, version, effective_date.	S3 + OpenSearch
Recommendation	Worker Twin output.	scenario, expected_range, confidence, drivers.	DynamoDB

10.1 Event sourcing rule

The system should avoid silently overwriting important worker events. For primary evidence events, store an append-oriented record with timestamp, source and content hash. Derived interpretations may be versioned separately. This makes later case reconstruction possible and reduces ambiguity about what the system knew at a particular time.

11. AWS Architecture



Service	Kavach responsibility	Why it is justified
Amplify Hosting	Web application delivery.	Fast live Ship It URL; clean deployment story.
Cognito	Worker authentication.	Identity boundary for sensitive records.
S3	Evidence artifacts and source documents.	Durable object storage; direct artifact handling.
API Gateway + Lambda	Application APIs and lightweight	Serverless execution and simple

	processing.	deployment.
DynamoDB	Worker ledger, cases, findings and state.	Predictable low-latency access and event-oriented entities.
OpenSearch	Policy/document retrieval and semantic search.	Search over source material and evidence metadata.
Bedrock	Model inference, multimodal extraction and reasoning.	Core AI substrate.
Strands Agents	Agent orchestration and tool use.	Visible AWS-native open-source AI learning.
Step Functions	Case lifecycle orchestration.	Long-running, retryable workflows.
EventBridge	Event-driven decoupling.	Connect ingestion, findings and notifications.
Cedar	Authorization.	Fine-grained, inspectable policy decisions.

11.1 Architecture principle

Every AWS service should map to a concrete responsibility. Do not add services solely because they are AWS-branded. This matters for the judging rubric: the architecture should be explainable in one sentence per service.

12. Feature Dependency Map

Capability	Requires worker ledger	Requires AI	Requires policy retrieval	Requires simulation
True Earnings	Yes	Yes	No	No
PayCheck	Yes	Yes	No	No
TripProof	Yes	Yes	No	Yes (for SLA)
Decision Review	Yes	Yes	Yes	Optional
Resolve	Yes	Yes	Optional	No
Worker Twin	Yes	Yes	No	Yes
GigOpt	Yes	Yes	No	Yes
Safety	Yes	Yes	Optional	Yes/No
ClaimKavach	Yes	Yes	Optional	No
Worker Voice	Yes	Yes	Optional	No

This dependency structure is intentional: if the team implements the worker ledger, evidence model, AI tool layer and core decision engine correctly, multiple visible features become thin product views over the same substrate.

13. Hackathon Scope: What Must Work

13.1 Non-negotiable MVP

- Worker workspace: Login, seeded worker profile and dashboard.
- Multimodal ingestion: Upload a payout screenshot + trip evidence; extract structured facts.
- Evidence graph: Show the source artifacts and events behind a conclusion.
- Earnings audit: Calculate expected vs actual payout and true-earnings view.
- Trip forensics: Reconstruct one disputed trip and quantify SLA consumption.
- Agent investigation: Supervisor + Earnings/Forensics/Policy specialist workflow.
- Policy retrieval: At least one curated policy/legal knowledge source with source references.
- Case generation: Generate a neutral, evidence-backed review package.

- Worker Twin demo: At least one scenario recommendation using historical data.
- AWS deployment: Live URL and working backend on AWS.

13.2 Stretch features

- Multilingual voice interaction
- Deactivation Shield full workflow
- Safety intelligence
- Accident/claim package
- Aggregated Worker Voice analytics
- Cross-platform comparison
- Advanced sensor fusion
- Cryptographic event signing beyond content hashing

Scope rule

A stretch feature is allowed only after the flagship investigation path is reliable. No stretch feature should break the primary demo or introduce a dependency the team cannot explain.

14. Four-Day Hackathon Implementation Strategy

Day	Primary outcome	Team-wide milestone	Do not spend time on
Thursday	End-to-end skeleton	Live URL, auth, database, upload, one Lambda, one Bedrock call, seeded worker/trip data.	Polishing advanced AI architecture before the first vertical slice exists.
Friday	Core intelligence works	Multimodal extraction, earnings audit, trip reconstruction, evidence graph, first agent tools.	Collective analytics, elaborate mobile telemetry.
Saturday	AI differentiation + architecture	Worker Twin, counterfactual simulator, policy retrieval, decision review; mentor feedback on AWS architecture.	New product areas that don't reuse the evidence graph.
Sunday	Reliability + narrative	End-to-end regression, deploy freeze, demo video, writeup and repository cleanup.	Last-minute architecture rewrites.

14.1 Four-person workstreams

Person / stream	Owns	Shared dependency
A — Product/UI	Dashboard, evidence explorer, case view, demo narrative.	Stable API contracts.
B — AI/Agents	Strands supervisor, specialist agents, tool contracts, evaluation fixtures.	Evidence and data schemas.
C — AWS/Backend	Lambda/API Gateway, DynamoDB, S3, Step Functions, EventBridge, deployment.	Agent tool APIs.
D — Data/Policy/Security	Seed datasets, policy documents, retrieval, Cedar, evaluation, privacy.	Evidence model and auth.

15. Three-Minute Demo Specification

Time	Screen	Message
0:00–0:20	Worker dashboard + penalty notification	“A platform deducted ₹350 for late delivery.”
0:20–0:45	Upload / ingestion	AI extracts payout, incentive and trip information.
0:45–1:20	Evidence graph + trip timeline	Kavach reconstructs the trip and identifies the source of the delay.
1:20–1:45	Agent run / finding card	Specialist agents disagree/check evidence; supervisor synthesizes a finding.
1:45–2:05	Counterfactual simulator	Show why the SLA was or was not feasible under observed conditions.
2:05–2:25	Case generator	One click produces an evidence-backed review request.
2:25–2:45	Worker Twin	“What should I do tomorrow?” → personalized scenario recommendation.
2:45–3:00	AWS architecture + final outcome	Show AWS services and return to quantified worker impact.

Demo rule

Every screen shown must be functional or a clearly seeded artifact produced by the same pipeline. The recording must never depend on a feature that exists only in the write-up.

16. UI / UX Specification

16.1 Main dashboard

KAVACH

REAL EARNINGS THIS WEEK ₹5,070
Platform payout ₹8,460
Effective hourly ₹95/hr

⚠ 3 issues detected
₹740 potential payout discrepancies
2 trips with conflicting delay evidence
1 incentive requiring review

[Explain my week] [Plan tomorrow] [Review cases]

16.2 Finding card

POTENTIAL DISCREPANCY
₹350 late-delivery penalty

Confidence 94%
Evidence 7 / 8 relevant signals
Missing platform internal dispatch time

WHY
7 min store wait → only 3 min remained in 10 min SLA

[View evidence] [Simulate] [Create review request]

16.3 Design language

- Evidence-first: claims visibly link to source artifacts.
- Minimal dashboard: focus on outcomes, not AI jargon.
- Neutral language: “potential discrepancy” rather than accusatory legal conclusions.
- Worker-centric wording: simple English plus Hindi/Kannada voice/text path for the demo if feasible.
- Confidence and missing evidence always visible for consequential findings.

17. Security, Privacy & Trust

Concern	Design response
Sensitive location	Use coarse location where precise coordinates are not necessary; restrict raw telemetry access.
Worker identity	Cognito authentication and least-privilege access.
Evidence export	Explicit action + authorization check; log export events.
AI hallucination	Structured tool outputs, evidence citations, deterministic calculations, missing-evidence field.
Legal overclaiming	Policy agent returns source references; findings use neutral categories.
Data aggregation	Worker Voice uses anonymized aggregated statistics and minimum group sizes.
Document integrity	Store hashes for important artifacts; never silently replace originals.

17.1 Threat model — minimum hackathon checklist

- Unauthorized user reading another worker’s evidence.
- Agent attempting to call a tool outside its role.
- Prompt injection inside uploaded documents.
- Model inventing a legal rule or financial result.
- Accidental exposure of raw GPS in an aggregated analytics view.
- Exporting more evidence than the worker intended.

18. Repository Source-of-Truth Structure

```
/docs
  PRODUCT.md           # canonical product definition
  ARCHITECTURE.md      # AWS + agent architecture
  DATA_MODEL.md       # entities, events, schemas
  AI_SPEC.md           # prompts, agents, tools, evaluations
  POLICY_SPEC.md       # policy sources + Cedar decisions
  DEMO.md              # exact demo scenario and expected outputs
  EVALUATION.md        # datasets, metrics, test results

/apps/web
/services/api
/services/agents
/services/calculation
/services/policy
/infrastructure
/policies
/data/seed
/evals
```

18.1 Canonical terminology

Term	Definition
Worker Ledger	Append-oriented record of worker events and state.
Evidence Graph	Links between events, artifacts, calculations, findings and cases.
Finding	Structured AI conclusion with confidence and evidence references.
Case	A worker issue requiring review or action.
Worker Twin	Personalized representation used for scenario reasoning and recommendations.
Discrepancy	A potential mismatch between expected/claimed and observed values; not automatically a legal violation.
Policy Agent	Retrieval-grounded AI component that explains a source policy or law.

19. Acceptance Criteria

Area	Definition of done
Authentication	User can sign in and only access their own seeded worker space.
Ingestion	At least two artifact types are parsed into structured records.
Audit	A known seeded discrepancy is detected correctly and tied to evidence.
Forensics	A disputed trip timeline is reconstructed and SLA feasibility calculated.
AI reasoning	Supervisor calls at least two specialist tools/agents and produces structured synthesis.
RAG	Policy answer cites the source document/version used.
Simulation	Worker Twin produces at least two scenarios and a recommendation.
Case	User can generate and view a neutral evidence-backed review package.
Authorization	At least one meaningful Cedar ALLOW/DENY path is demonstrated.
Deployment	Public AWS-hosted application and working backend are available for judging.
Demo	Three-minute recording demonstrates the full primary path without manual database edits.

20. Risks & Design Responses

Risk	Severity	Response
Trying to build too many disconnected features	High	Shared evidence graph and strict MVP vertical slice.
Agent unreliability	High	Structured tools, deterministic math, evidence citations, evaluation set.
Legal claims overreach	High	Neutral finding labels and source-grounded policy agent.
Telemetry implementation complexity	High	Start with seeded event traces and simple phone/browser events; advanced sensor fusion is optional.
AWS integration surprises	Medium	Deploy the skeleton early; keep service boundaries simple.
Demo data looks fake	Medium	Use realistic, internally consistent trip/payout artifacts and clear source labels.

Privacy concerns undermine trust	High	Least-privilege access, coarse data where possible, visible privacy boundaries.
----------------------------------	------	---

21. Post-Hackathon Expansion

- Platform connectorsOpt-in ingestion from supported worker data exports or notifications rather than screen scraping.
- Cross-platform intelligenceCompare worker outcomes across platforms without exposing private data.
- Worker cooperative analyticsAllow worker organizations to view anonymized patterns.
- Insurance / claims integrationsConnect evidence packages to supported claim workflows.
- Real-time edge telemetryAdd richer sensor fusion only after privacy, consent and device reliability are solved.
- Policy updatesVersion policy sources and alert workers when rules change.
- Personalized financial planningExtend Worker Twin into weekly income planning and emergency-buffer scenarios.

22. Research & Reference Notes

The following references are used to ground the problem framing and technology choices. The product specification remains the canonical project document; external sources should be treated as supporting evidence, and implementation details should be rechecked against current AWS documentation during development.

[R1] AWS Bedrock — Multi-agent collaboration documentation. Amazon documents supervisor/collaborator patterns for complex tasks and specialized roles. <https://docs.aws.amazon.com/bedrock/latest/userguide/agents-multi-agent-collaboration.html>

[R2] Strands Agents documentation — SDK for building, managing, evaluating and deploying AI agents; AWS Bedrock is the default model provider in the quickstart. <https://strandsagents.com/docs/user-guide/quickstart/overview/>

[R3] ILO Digital Labour Platform Tracker — Karnataka Platform Based Gig Workers (Social Security and Welfare) Act, 2025; includes transparency obligations around automated monitoring and decision-making systems. <https://digitallabour.ilo.org/legislation/karnataka-platform-based-gig-workers-social-security-and-welfare-act-2025>

[R4] AWS Prescriptive Guidance — Cedar overview; policy-based authorization with ALLOW/DENY evaluation and separation of authorization logic from business logic. <https://docs.aws.amazon.com/prescriptive-guidance/latest/saas-multitenant-api-access-authorization/cedar.html>

[R5] NITI Aayog — India’s Booming Gig and Platform Economy; projected gig workforce expansion to approximately 23.5 million by 2029–30. <https://www.niti.gov.in/node/375>

[R6] User-provided Gemini Deep Research document — “Algorithmic Accountability in Quick-Commerce Logistics / KavachRider” supplied as project research input. It informed the original worker-side telemetry, policy-as-code, dispute and AWS architecture concepts.

23. Decision Log & Open Questions

Decision	Rationale
Ship It is the target track	Maximizes opportunity to demonstrate architecture and AWS-native engineering.
Worker-side operating layer is the core product	Allows broad capabilities without making the problem vague.
Evidence graph is the shared substrate	Avoids disconnected feature sprawl and makes AI outputs auditable.
Agents are specialists, not theater	Each agent must have a distinct responsibility and tool boundary.
Legal certainty is not a product promise	Reduces risk and keeps conclusions evidence-grounded.
Advanced telemetry is stretch	Simple seeded traces are enough for the core hackathon demo.

23.1 Open questions to resolve during research/build

- Which exact worker evidence sources are easiest and safest to use for the demo?
- Which payout/incentive schemas should the seeded dataset emulate?
- Which policy sources should be treated as authoritative and versioned?
- What minimum anonymization threshold should be required for collective analytics?
- Which Bedrock model gives the best quality/latency/cost trade-off under the event credits?

24. Final Product Definition

One sentence

Kavach AI is an independent worker intelligence layer that reconstructs a gig worker's work from their own evidence, audits earnings and platform decisions, predicts better choices, and turns findings into actionable protection.

What success looks like

The worker understands the numbers.

They can see real earnings, costs, waiting and discrepancies.

The worker understands the event.

They can inspect a trip timeline and see what evidence supports it.

The worker can interrogate the black box.

The AI can explain likely causes of a payment or decision using available evidence.

The worker can simulate alternatives.

The Worker Twin helps answer what to do next and why.

The worker can act.

The system can generate an evidence-backed review package.

The judges can see the engineering.

AWS services and agent/tool boundaries are visible and purposeful.

The hackathon version should feel like a small but real piece of worker infrastructure: one coherent source of truth, one AI investigation engine, and several compelling views over the same evidence. That combination is the intended ceiling for the project.